FINAL SENIOR RESEARCH LAB PRESENTATION: JMESSAGE

Sam Stankiewicz & Nidhish Shakya

Dr. Yilmaz, Pd. 3, 5/20/2026

PROBLEM

AMBIGUITY

Communicating with other students is bothersome without immediate directory of classmates.

PHONES

Phones, the default medium of communication, are banned. This leaves a need for a new method of communication.

MESSENGER & GMAIL

The primary substitutes for text, Gmail and Messenger, are both poorly adapted for TJ-style communication

Background

- Messaging App
  - Updating front end - matches data stored on back end
  - Encrypting occurs during this transfer of data between UI
  - Apps like messenger use a similar framework on a larger scale
- Ion OAuth
  - Allows for log in via Ion
    - Get, accurate, user data
- Will be hosted on Sam's personal domain (trulycalc.com)
  - TJ Director didn't end up working out

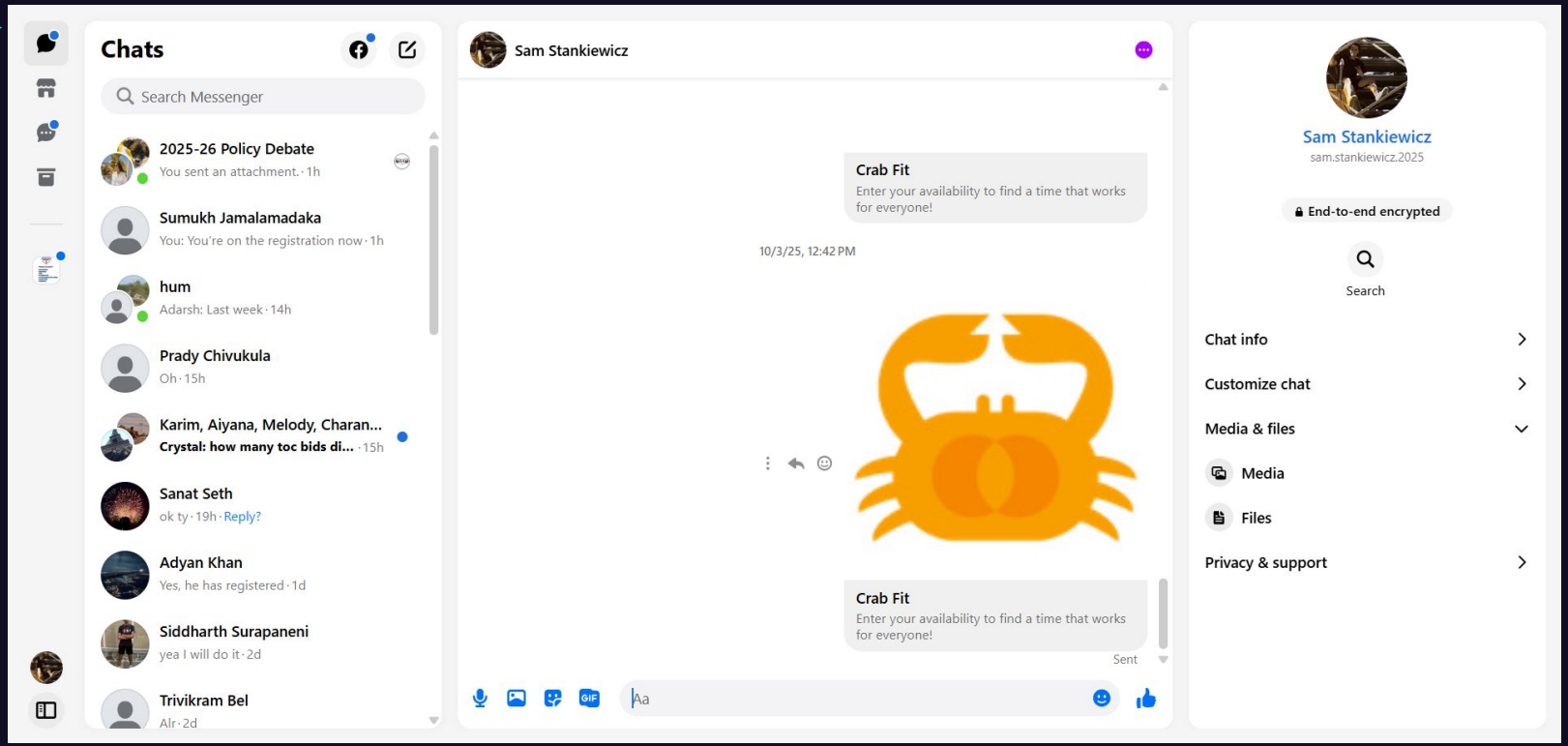
Pre Existing Solutions

MESSENGER

- Make chats with others
  - Group chats available
- Somewhat quick/convenient when looking up someone to chat to
- Blocked on school computers by default

iMESSAGE

- High convenience, but requires a phone
  - Usually also requires either mobile data
- Require a phone number - inconvenient
  - Could be very personal, privacy concerns



Messenger - Demo (10/6/2025)

iMessage - Demo (10/6/2025)

NOVELTY

FOR & BY TJ

Able to use TJ-specific information to make a platform that works for a TJ student workflow

ION INTEGRATION

Uses Ion integration to include a complete directory of all TJ students. Allows for secure communication with any TJ student.

Ease Of Search

Allows user to find and contact others in TJ with ease. Let's people find others in your class to communicate easily.

USE IN SCHOOL

Because our app is designed with a limited scope & function, it's ideal for use in school.

IMPACT

New, more seamless messaging apps for TJ students mean that communication becomes easier across grades and schedules. For some of the busiest students in the country, any gain is impactful

- Better networking amongst students
- Better communication within clubs
- Improved information sharing between students in a class

For TJ students, this jump in communication capability could be impactful and notable.

Why Is Ours Better?

Ion-Based login

Allows us to store data like name; allows for other users to quickly look up classmates.

Additions

Possible additional features such as auto creation of group chats based on a user inputted schedule

Accessibility

All students can use it - no need to make an account, just log in through pre existing ion account

General Methodology

Database

We're using MongoDB with Redis for caching

- Document-based format for messaging system
- Better read/write efficiency at scale vs Firebase

Framework

NodeJS/ExpressJS framework used

- Faster than Flask
- Easier to build than NextJS

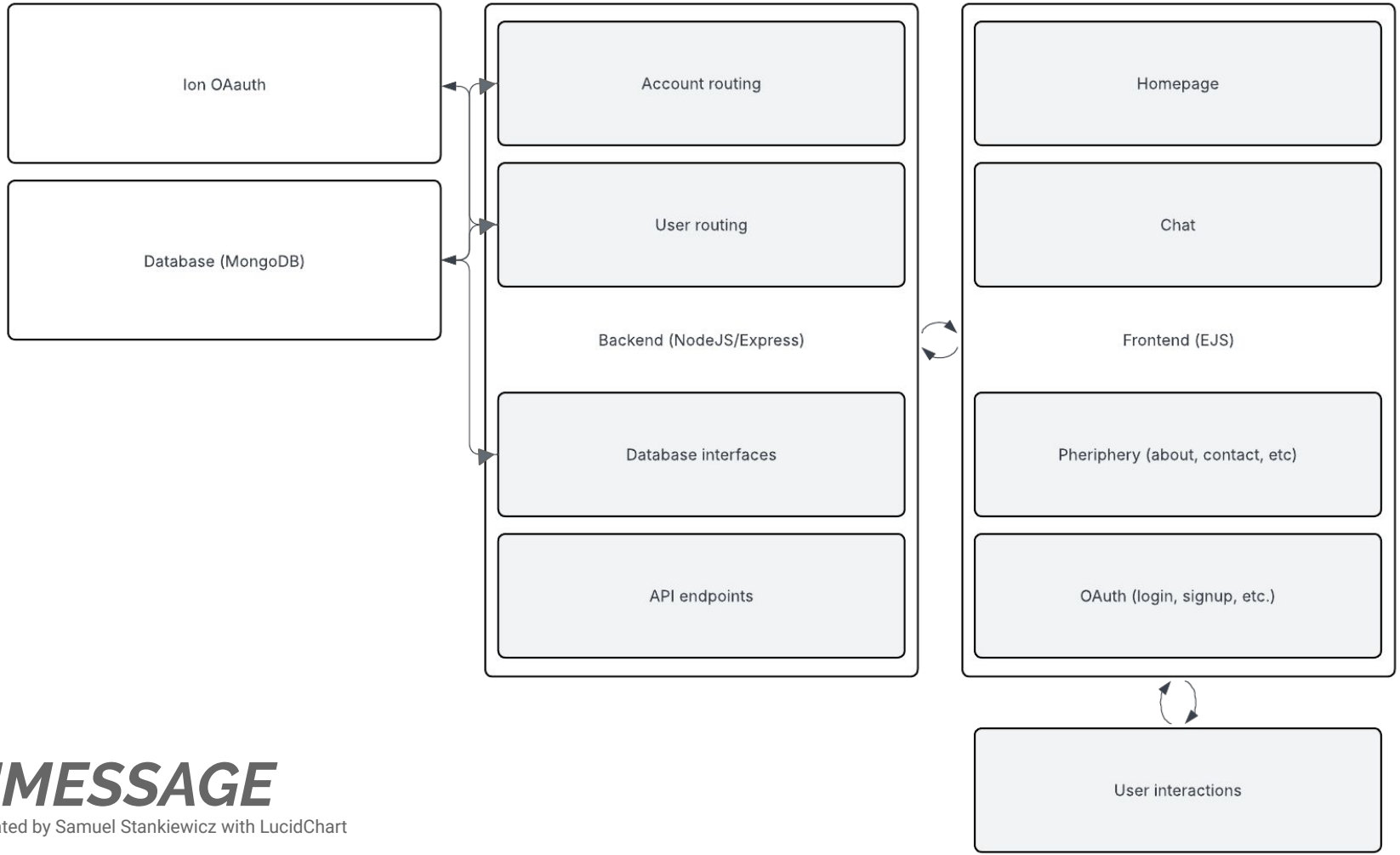
Ion OAuth

Use pre existing OAuth code from sysadmins to integrate Ion accounts

- Limits to TJ students
- Improves signup/login efficiency

USE IN SCHOOL

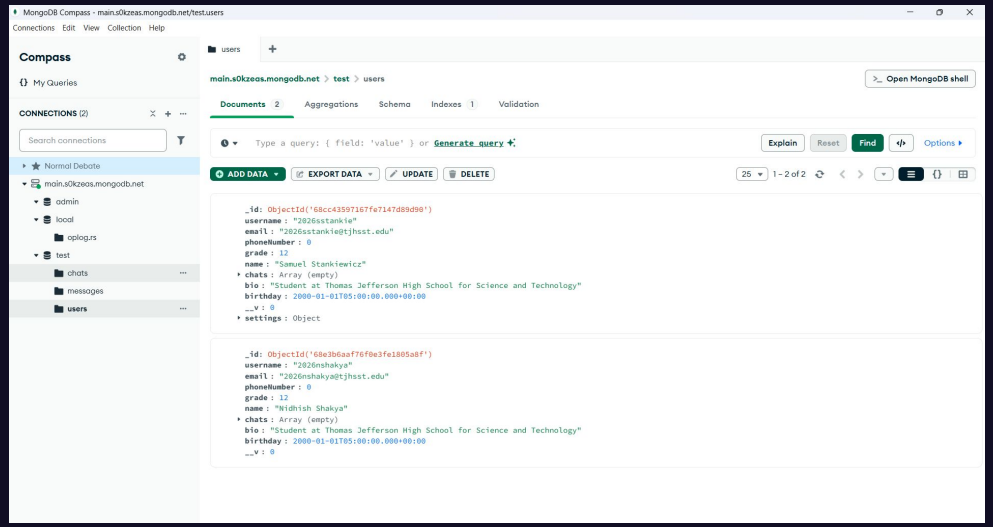
Because our app is designed with a limited scope & function, it's ideal for use in school.



JMESSAGE

Created by Samuel Stankiewicz with LucidChart

Results - Login



Screenshot from MongoDB, Sam Stankiewicz

- Fully completed log in & storage of user information
  - Users stored as objects within MongoDB
  - Login information stored directly through ion
  - Ion OAuth verifies user upon login - we DON’T see your password

Cont.

MongoDB Compass - main.s0kzeas.mongodb.net/test.users

Connections Edit View Collection Help

Compass

My Queries

CONNECTIONS (2)

Search connections

Normal Debate

main.s0kzeas.mongodb.net

admin

local

oplog.rs

test

chats

messages

users

users

main.s0kzeas.mongodb.net > test > users

Open MongoDB shell

Documents 2 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or Generate query

Explain Reset Find Options

ADD DATA EXPORT DATA UPDATE DELETE

25 1 - 2 of 2

```
_id: ObjectId('68cc43597167fe7147d89d90')
username : "2026sstankie"
email : "2026sstankie@tjhsst.edu"
phoneNumber : 0
grade : 12
name : "Samuel Stankiewicz"
chats : Array (empty)
bio : "Student at Thomas Jefferson High School for Science and Technology"
birthday : 2000-01-01T05:00:00.000+00:00
__v : 0
settings : Object
```

```
_id: ObjectId('68e3b6aaf76f0e3fe1805a8f')
username : "2026nshakya"
email : "2026nshakya@tjhsst.edu"
phoneNumber : 0
grade : 12
name : "Nidhish Shakya"
chats : Array (empty)
bio : "Student at Thomas Jefferson High School for Science and Technology"
birthday : 2000-01-01T05:00:00.000+00:00
__v : 0
```

Screenshot from MongoDB, Sam Stankiewicz

Results - Chats

```
_id: ObjectId('69de7080d9ef681c0fe34a46')
▸ members : Array (2)
  createdAt : 2026-04-14T16:51:12.856+00:00
▸ media : Array (empty)
  name : "test chat"
  picture : null
  chats : 0
▸ read : Array (empty)
▸ messageIds : Array (2)
  __v : 0
```

Screenshot from MongoDB, Nidhish Shakya

- Chats = Objects in MongoDB
- Chats contain Ids for message “blocks”
  - Points towards chunks of messages that belongs to the specified chat
- When loading, server looks for all the chats the user is in
  - Messages are only loaded once the user opens a specific chat

Results - Messages

```
_id: ObjectId('69ea2e808fd6334cc2a4901d')
fill : 19
messages : Array (19)
  0: Object
    sender : "2026nboukens"
    senderName : "Nick Boukens"
    text : "YO"
    timeSent : 2026-04-23T14:36:59.255+00:00
  1: Object
  2: Object
    sender : "2026nshakya"
    senderName : "Nidhish Shakya"
    text : "hey nick"
    timeSent : 2026-04-23T14:37:02.962+00:00
  3: Object
  4: Object
  5: Object
```

Screenshot from MongoDB, Nidhish Shakya

- Messages = Object
  - Stores message that was sent
  - User who sent it
  - Date
- Messages are stored in “chunks” of 50
  - More space efficient, and makes loading a chat faster
  - Initially only load most recent chunk

Cont.

```
_id: ObjectId('69ea3b333e936aa9a9017f5b')
fill : 2
▾ messages : Array (2)
  ▾ 0: Object
      sender : "2026nboukens"
      senderName : "Nick Boukens"
      text : "hello"
      timeSent : 2026-04-23T15:31:17.382+00:00
  ▾ 1: Object
      sender : "2026nshakya"
      senderName : "Nidhish Shakya"
      text : "hello"
      timeSent : 2026-04-23T15:31:21.873+00:00
index : 0
```

Screenshot from MongoDB, Nidhish Shakya

Results - Searching

```javascript
async function getNames () {
    try{
        const response = await fetch('/testSearch', {
            method: 'POST'
        });
        const result = await response.json();
        return result
    } catch (err) {
        console.log(err);
    }
    return 0
}
```

```javascript
app.post('/testSearch', async (req, res) => {
    try{
        let aggUsers = [];
        await User.find({}, {name:1, _id:0})
            .then(users => {
                forEach(users, (user) => aggUsers.push(user['name']))
            });
        return res.send(aggUsers);
    } catch (err) {
        console.log(err);
    }
});
```

Screenshot from VSCode, Nidhish Shakya

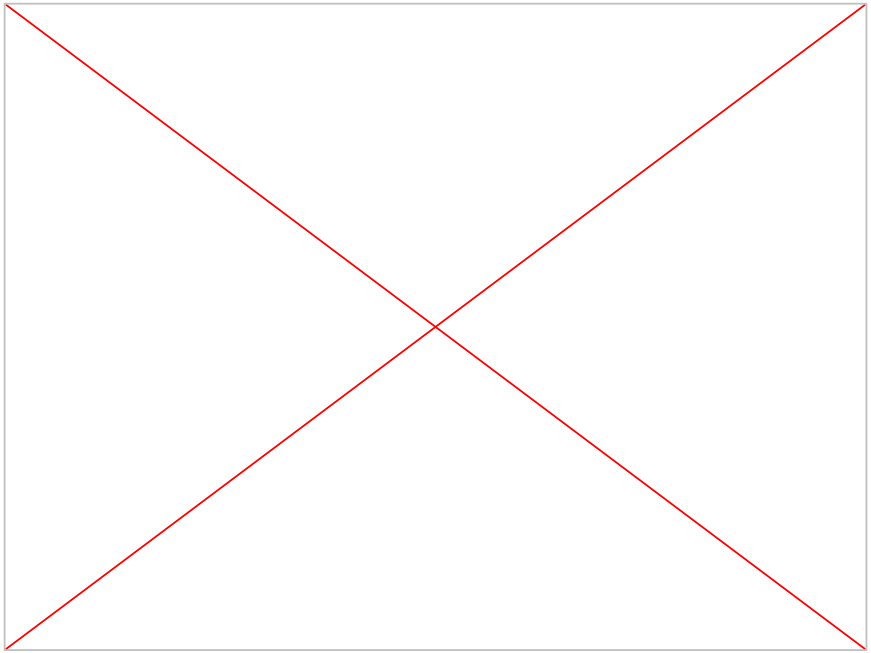
- Look through MongoDB for all templates that match a schema
    - In this case the schema will be the one we use for users
- Aggregate the names and ids found on that matched template
- Send it back to our javascript to update the html

Progress Since Last Presentation

Project's getting there, all the basic features and use cases are implemented, and is most definitely usable, if nothing else. In all, app fully functional as a messaging app, but still quite rough around the edges.

- Messaging fully functional
- Chats fully functional
- Group chats *functional*
- Webhooks fully implemented
- App successfully hosted
- Added sanitation of messages

DEMO



Recording from JMessage application, Nidhish Shakya. 4/23/2026.

Limitations

Limited Usage

Since our app is based on Ion, there is no way for outside users to use it

Lack of Features

Could lack features that are common in other apps - we are only two people after all

Time

Basic features are still yet to be fully integrated, we're running into issues with time.

CONCLUSIONS AND FUTURE WORK

A lot of the basic work is done, but, nonetheless, there's two major things left, a better/more intuitive searching and actual chat encryption. Additional features like image/emoji compatibility or a reply function if time allows.

Important features to be added before TJStar

- Encryption for messages
- Better overall design and UX
- Better/More intuitive searching
  - Also more intuitive way to create a group chat

Additions/Quality of Life

- Image compatibility and emojis
- Reply function
  - Highlight or redirect to a specific message you're replying to
- Notification system

REFERENCES

[1] N. Kiran, “Securing Chat applications: Strategies for end-to-end encryption and cloud data protection,” World Journal of Advanced Engineering Technology and Sciences, vol. 13, no. 2, pp. 528–540, Dec. 2024, doi: https://doi.org/10.30574/wjaets.2024.13.2.0634.

[2] H.-Y. Wang, “A Review of Instant Messaging and Mobile Messaging Applications,” International Journal of Economics, Finance and Management Sciences, vol. 7, no. 1, p. 13, 2019, doi: https://doi.org/10.11648/j.ijefm.20190701.13.

[3] MongoDB, “MongoDB Documentation,” https://github.com/mongodb/docs-bi-connector/blob/DOCSP-3279/source/index.txt, 2024. https://www.mongodb.com/docs/

[4] MongoDB, “MongoDB Documentation,” https://github.com/mongodb/docs-bi-connector/blob/DOCSP-3279/source/index.txt, 2024. https://www.mongodb.com/docs/

[5] “Express 5.x - API Reference,” expressjs.com. https://expressjs.com/en/5x/api.html

Q&A

Any questions?

THANKS!